

ADR-005 — Gestion des erreurs normalisée et stratégie de versionnage

Champ	Valeur
Statut	Accepté
Date	2026-02-01
Auteurs	Pascal Bourgoïn

Contexte

Avec trois microservices distincts exposant des endpoints REST, deux problèmes se posaient rapidement :

1. **Erreurs incohérentes** : sans convention partagée, chaque service pouvait retourner ses erreurs dans un format différent (stack trace Spring par défaut, message HTML, ou objet JSON maison). Les clients auraient eu à gérer plusieurs formats d'erreur.
2. **Versionnage des routes** : comment distinguer les endpoints de chaque service quand tous passent par la même API Gateway ? Plusieurs approches sont possibles : préfixe `/v1/` dans l'URL, header `Accept-Version`, ou préfixe par service dans le chemin.

Décision

Erreurs : objet `ErrorResponse` partagé via le module `common`

Un `GlobalExceptionHandler (@RestControllerAdvice)` dans le module `common` intercepte toutes les exceptions non gérées et retourne toujours un objet JSON de la forme :

```
{
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Insufficient funds on account 1596875651",
  "timestamp": "2026-03-08T14:23:10.452Z"
}
```

Mapping des exceptions vers les codes HTTP :

Exception	Code HTTP
<code>MethodArgumentNotValidException</code>	400
<code>ClientNotFoundException</code> , <code>PaymentNotFoundException</code> , <code>AccountNotFoundException</code>	404

Exception	Code HTTP
<code>BadCredentialsException</code>	401
Doublon email / NAS	409
<code>InsufficientFundsException</code>	422
Exception non gérée	500

Versionnage : préfixe de service dans le chemin

Plutôt qu'un préfixe `/v1/` générique, chaque service utilise son propre préfixe qui sert aussi de namespace :

- `identity-service` → `/identityservice/**`
- `account-service` → `/accountservice/**`
- `payment-service` → `/paymentservice/**`

KrakenD expose ces routes sous `/api/**` sans le préfixe de service. Les clients n'interagissent jamais directement avec les chemins internes.

Ce choix évite d'avoir à gérer la version dans KrakenD pour l'instant. Si une v2 est nécessaire, il suffirait d'ajouter `/identityservice/v2/**` dans le service concerné et un nouvel endpoint dans `krakend.json`.

Conséquences

Positives

- Format d'erreur prévisible pour les clients — testé dans le smoke test (7 cas d'erreur validés : 400, 401, 404, 409, 422).
- Le module `common` force la cohérence entre les 3 services sans dupliquer le code.
- Le préfixe de service rend les logs et les métriques Prometheus faciles à filtrer (`uri=~"/paymentservice/.*"`).

Négatives / Limites

- Pas de versionnage `/v1/` explicite dans les URLs exposées par KrakenD — si une rupture de compatibilité est nécessaire, il faudra planifier la migration des clients.
- Le `GlobalExceptionHandler` masque les stack traces en production ce qui est voulu, mais en développement il faut regarder les logs du service plutôt que la réponse HTTP.